CSE 311 Artificial Intelligence

Project Evaluation Prep Guide

Explainable Sudoku AI Coach

A teaching website that uses constraint reasoning to explain every move

Prepared 9 October 2026 from the 9/10/26 presentation schedule (IIIT Kottayam, Dr. Sushitha Susan Joseph) and a review of the code in the Sudokusolver repository, including measurements taken on the current code.

What's inside	
1. Evaluation format	How the 20-minute slot is scored
2. What other groups are doing	The 9 Oct line-up and how we stand out
3. The website at a glance	What a user can do, and how the system is built
4. The AI inside	Each feature explained as an AI technique, mapped to the syllabus
5. Implementation and numbers	Modules, measured speeds, tests
6. The 15-minute plan	Timing, speakers, live demo script, fallback plan
7. Q&A bank	30 likely viva questions with short answers
8. Fix before the evaluation	Things in the repo an examiner could catch
9. Presentation skills	How to score well on delivery

1. Evaluation format (from the schedule)

Item	Detail
Total slot per group	20 minutes, strictly time-boxed (slots run back to back)
Presentation + Demo	15 minutes
Q&A / Evaluation	5 minutes
Scored on	**Methodology**, **Implementation**, **Technical Q&A**, **Presentation Skills**

Note: our project is **not** on the 9 Oct (Fri) schedule. Only 6 groups are listed, so we are most likely in a later session. Confirm the date and slot with the faculty or class representative. If you can, sit in on the 9 Oct presentations and note the kinds of questions Dr. Joseph asks.

What each component probably rewards

Component	What the evaluator wants	Our strongest evidence
Methodology	A clear problem formulation and a justified choice of AI technique	Sudoku as a CSP; a rule-based technique engine (9 human techniques in 3 tiers); forward chaining for hints; speculative propagation for What-If
Implementation	Working software, a live demo, sound engineering	A complete FastAPI and web app; 39 automated tests; every hint computed live in about 2 ms
Technical Q&A	Every member understands every part	Section 7: everyone should be able to answer all 30 questions
Presentation Skills	Timing, structure, clarity, a smooth demo	Section 6 plan; the demo is the centrepiece of a teaching website

For a teaching website, the demo *is* the project. Plan on about **6 minutes of slides and 8 minutes of live demo**, with a 1-minute buffer.

2. What other groups are doing (9 Oct schedule)

Titles and team members are from the schedule. The "Likely AI core" column is a *guess from the title only*.

Slot	Project	Team	Likely AI core (inferred)
11:30-11:50	QUILL: learning-based system-wide autocomplete for Windows	Anita George, Lestlin Robins, Athul Krishna P S, Kiran J	Language modelling and next-word prediction
11:50-12:10	Gesture-Based Video Control	Mandhadi Snigdha Reddy (solo)	Computer vision: hand-landmark detection and gesture classification
12:10-12:30	Personal AI Finance Advisor	Krishnendu Rajesh, Sarah Mariam Sunil, Nidhishree Renukaprasad, Reshmitha Garika, Mathew George	LLM or recommendation system; expense classification
12:30-12:50	Multi-Hop RAG System	Sarthaki Bhoir, Theertha S, Dhanvanth Balaji, Anshul Goyal	Retrieval-augmented generation: embeddings, vector search, multi-step retrieval
12:50-1:10	Semantic Caching Layer for LLM APIs	Tejendra Dronadula, Bharani B, Pavan Naik Vadite, Sam Raj Gella	Embedding similarity to reuse LLM answers
1:10-1:30	AI-Based Verilog HDL Assistant	Palam Chaitanya Reddy, D. Durga Sai Charan Teja, Manikonda Vineeth Kumar Reddy, Polisetti S V N A Manikanta	LLM code generation for hardware design

What this tells us

- Most of the 9 Oct groups build on **machine learning or LLMs**. Ours is a **symbolic AI** project: constraint reasoning and a rule-based inference engine. That makes it stand out.

- **Our pitch against LLM tutors:** "An LLM can explain Sudoku, but it can invent a deduction that isn't true. Every hint our coach gives comes from a rule that actually fires on the current board, so it is always correct and can always be traced back. It is explainable by construction."
- **Expect:** "Why not just use ChatGPT?" and "Where's the AI? There's no learning." The answers are Q1 and Q2 in Section 7.
- The RAG and caching groups will likely show numbers such as hit rate and latency. We should also show numbers: hint speed, generator speed, technique coverage and tests (Section 5).

3. The website at a glance

One-line pitch: a Sudoku tutor that adapts to the learner's level and explains every deduction the way a human expert would, using the same constraint reasoning a CSP solver uses.

3.1 What a user can do

Step	Feature	What happens
1	Choose a level or take the skill check	Beginner, Intermediate or Advanced. Unsure users take a 4-question check that recommends a track.
2	Beginner lessons	7 short interactive lessons: the one rule, reading the grid, full house, hidden single, naked single, pencil marks, then a first guided puzzle. Each is a short explanation, a task on the board, and instant feedback.
3	Get a puzzle	Choose from a graded puzzle bank (17 puzzles: 6 beginner, 6 intermediate, 5 advanced), generate a fresh puzzle at your level, or load your own.
4	Hint ladder ("Need a hint?")	Stage 1 **Nudge** (where to look), Stage 2 **Technique** (its name, a definition and the key cells), Stage 3 **Reveal** (the answer with a step-by-step walkthrough). If the board contains a mistake, the coach points that out before giving any hint.
5	Check square	Tests a move: a rule conflict, a move that is legal but wrong (a dead end), or a correct move. A correct move comes with an explanation of why it works.
6	What-If	Tries a hypothetical move on a copy of the board and shows the chain of consequences, including any contradiction. The real board is not changed.
7	Undo, Reset, Solve, Progress	Move history; full solve by the CSP solver; a progress panel listing the techniques encountered and curriculum progress.

3.2 Architecture

Layer	Files	Role
Web UI	static/index.html, app.js, style.css	Level select, skill check, lessons, 9×9 board, number pad, coach panel
REST API	api/app.py (FastAPI)	Endpoints such as /api/teacher/hint-ladder, /api/teacher/check-move, /api/what-if, /api/solve
Teaching logic	hint_ladder, move_checker, what_if, lesson_engine, skill_check, coach_session	Turns reasoning into feedback a learner can follow
Human-technique engine	api/techniques/*	9 deduction rules in 3 tiers; finds the next step or solves the whole puzzle by logic alone
Puzzles	puzzle_generator, puzzle_bank(_builder), generate_dataset.py	Generates unique puzzles graded by the techniques they need; stores the graded bank
Reasoning trace	reasoning_trace.py, board_state.py	Constraint propagation that records each step (ReasoningStep), board state with undo
CSP engine	src/board.py, csp_solver.py, validator.py	Cells, units and peers; Norvig propagation with MRV search; an independent solution checker

4. The AI inside: know this cold

4.1 Sudoku as a CSP (the foundation)

Element	Definition
Variables	81 cells, A1 to I9
Domains	{1..9} for empty cells. In the app these are the **candidates** ("pencil marks").
Constraints	27 AllDifferent units (9 rows, 9 columns, 9 boxes). Each cell has **20 peers** it must differ from.
Key idea	Every human technique is a rule that **removes values from candidate sets** (domain reduction), which is the same thing constraint propagation does in a CSP.

4.2 Feature-by-feature: the AI technique behind each one

Feature	AI technique	How it works (in one breath)
Candidates and propagation	Constraint propagation (Norvig's rules)	Rule 1: a placed digit is removed from all 20 peers. Rule 2: if a unit has only one place left for a digit, the digit goes there. Each step is recorded as a ReasoningStep and marked *deduced* or *guessed*.
Technique engine	Rule-based inference (a production system)	9 rules tried from easiest to hardest: full house, naked single, hidden single, naked pair, hidden pair, pointing pair, box-line reduction, naked triple, X-Wing. The first rule that fires is the next step. Easiest-first mimics how a human solves.
Hint ladder	Forward chaining with explanation	Elimination rules don't place a digit on their own. The engine chains steps forward, carrying eliminations, until a placement is unlocked. It teaches the **hardest technique in that chain** and reveals the result in 3 stages. The user's level caps which techniques are used.
Mistake detection	Constraint checking against ground truth	Before hinting, the coach scans for peer conflicts and for cells that differ from the unique solution.
Check square	Constraint check + solver as oracle	Case A: a peer already has the digit. Case B: legal now, but differs from the unique solution (found by the CSP solver), so it must lead to a contradiction later. Case C: correct, with an explanation of why it works.
What-If	Hypothetical reasoning (lookahead)	Assigns the value on a **deep copy**, runs propagation, and reports the cascade and any domain wipe-out. This is the same idea as forward checking and propagation in backtracking search.
Puzzle generator	Generate-and-test search	Build a random full grid by randomised backtracking. Blank cells one at a time, keeping a removal only if the puzzle stays solvable *by techniques at or below the target tier*. Accept only if it actually *needs* that tier.
Solve	Backtracking search with MRV + propagation	Norvig's agent: propagate, then branch on the cell with the fewest candidates (MRV).
Skill check / level	Simple user model (rule-based classifier)	Score plus self-reported experience gives Beginner, Intermediate or Advanced. The level controls lessons, puzzles and which hint techniques are used.

4.3 Map it to the AI syllabus (say this out loud)

- **Intelligent agent:** a tutoring agent. Its percepts are the board and the user's actions; its actions are hints, feedback and puzzles; its goal is to help the user learn. The environment is fully observable, deterministic and discrete.
- **CSPs:** variables, domains and constraints; constraint propagation; MRV; backtracking search (Russell & Norvig, Ch. 6).
- **Knowledge representation and inference:** techniques written as if-then rules over candidate sets, applied by forward chaining.
- **Search:** randomised backtracking for grid generation; generate-and-test for grading.
- **Explainable AI (XAI):** each output can be traced to the rule and the cells that caused it.
- **Adaptivity:** user level controls the curriculum and the technique ceiling for hints.

4.4 The techniques in one line each

Tier	Technique	Rule
Beginner	Full house	A unit has one empty cell, so it gets the missing digit
Beginner	Naked single	A cell has only one candidate left
Beginner	Hidden single	A digit has only one possible cell in a unit
Intermediate	Naked pair	Two cells in a unit share the same 2 candidates, so remove those digits from the rest of the unit
Intermediate	Hidden pair	Two digits fit only in the same 2 cells of a unit, so remove other candidates from those cells
Intermediate	Pointing pair	Within a box, a digit is confined to one row or column, so remove it from that line outside the box
Intermediate	Box-line reduction	Within a row or column, a digit is confined to one box, so remove it from the rest of that box
Advanced	Naked triple	Three cells in a unit hold only 3 digits between them, so remove those digits from the rest of the unit
Advanced	X-Wing	A digit is confined to the same 2 columns in 2 rows, so remove it from those columns in all other rows (or the same with rows and columns swapped)

5. Implementation and numbers

5.1 Tech stack

Python 3, FastAPI + uvicorn (REST API), pydantic (request models), plain HTML/CSS/JavaScript (UI), unittest (tests). The backend is about 4,000 lines and the front end about 3,600 lines.

5.2 Measured on the current code (9 Oct 2026, this laptop)

Measurement	Beginner	Intermediate	Advanced
Puzzles in the graded bank	6	6	5
Clues per puzzle	32-38	22-26	24-25
Techniques used across the tier's puzzles	Full house, naked single	+ hidden single, naked pair, hidden pair	All 9, including pointing pair, box-line reduction, naked triple and X-Wing
Full hint (Stage 3) generation, median	2.0 ms	2.1 ms	2.1 ms
CSP solve, median (search nodes)	1.8 ms (0)	1.8 ms (2)	1.9 ms (3)
Fresh puzzle generation (one run, seed 11)	0.11 s, 33 clues	0.34 s, 24 clues	1.77 s, 25 clues

Advanced puzzles are rare in random generation, so a background pool keeps a few ready (data/generated_pool.json).

5.3 Testing

Suite	Tests	Covers
tests/test_solvers.py	4	Board geometry (27 units, 20 peers), parsing, validator, CSP solver configurations
sudoku_ai_coach/tests/*	35	API endpoints, hint ladder stages, puzzle bank, generator (uniqueness, grading), teaching-quality checks, each technique detector
Total	**39, all passing**	Run: `python -m unittest discover -s sudoku_ai_coach/tests -t .`

5.4 Key design decisions (good to mention)

- **Correct first, then teach:** hints come only from rules that actually fire, and Case B is judged against a solution the CSP solver found and the validator checked.
- **Teach the hardest step, not the last step:** the hint names the key technique in the chain, so the learner learns the idea that unlocked the cell.
- **Mistakes first:** giving a forward hint on a board that already contains an error would teach the wrong thing.
- **Never spoil early:** three stages, with the answer only at Stage 3.
- **Grade by technique, not by clue count:** a 24-clue puzzle can be easy, and a 30-clue puzzle can need an X-Wing.

6. The 15-minute plan

This assumes 4 speakers (M1-M4). Replace them with real names, and have each person rehearse a backup part as well.

Time	Segment	Speaker	Key message
0:00-1:00	Hook	M1	"Sudoku apps give you the answer. We built an AI that teaches you *why*, and is never wrong about it."
1:00-2:30	Problem + CSP formulation	M1	81 variables, 27 AllDifferent units, 20 peers. Candidates are domains, and techniques reduce domains.
2:30-4:30	The AI inside	M2	Rule-based technique engine (9 rules, 3 tiers), forward-chained hints, What-If lookahead, generate-and-test puzzles.
4:30-5:30	Architecture	M3	UI, FastAPI, teaching logic, technique engine, CSP engine. Show the numbers: about 2 ms per hint, 39 tests.
5:30-13:30	**LIVE DEMO**	M3 drives, M4 narrates	Follow the script below.
13:30-14:30	Limitations + future work	M4	Single board per server, no user study yet, techniques stop at X-Wing. Future: more techniques, accounts, a learning-gain study.
14:30-15:00	Conclusion	M1	"Symbolic AI gives explanations that are correct by construction." Thank the examiner and invite questions.

Live demo script (about 8 minutes)

#	Action on screen	What to say
1	Open the site and click "Not sure? Take a 2-minute check". Answer the 4 questions.	"The coach places the learner first. That is a simple user model."
2	Pick Beginner and go through 1-2 lessons (e.g. Hidden Single).	"Each lesson is an explanation, a task and instant feedback. The rule it teaches is the same rule the AI uses."
3	Change level to Intermediate and load a bank puzzle.	"Puzzles are graded by the hardest technique they need, not by clue count."
4	Need a hint: Stage 1, then 2, then 3.	"Stage 1 is a nudge, Stage 2 names the technique, Stage 3 reveals the answer with a walkthrough. Here a naked pair unlocks the cell."
5	Check square: enter a peer conflict (Case A), then a legal but wrong digit (Case B), then the correct digit (Case C, Why does this work?).	"The three cases are a rule break, a dead end and a correct move, and a correct move is always explained."
6	Place a wrong digit on the board, then ask for a hint.	"The coach catches the mistake before it gives a hint."
7	What-If on a cell with a wrong value.	"It runs propagation on a copy of the board and shows the contradiction. The real board is never changed."
8	New puzzle at Advanced, then a hint showing X-Wing or a pointing pair.	"The puzzle was generated live and graded by our technique engine."
9	Solve.	"The CSP solver finishes it in about 2 ms, using propagation plus MRV search."

Fallback plan: (1) Start the server before your slot: `python -m uvicorn sudoku_ai_coach.api.app:app --port 8000`. (2) Check that fastapi, uvicorn and httpx are installed on the presenting laptop, and test the page at projector resolution. (3) Note down the Case A, B and C cells and digits for your demo puzzle *in advance*, so you don't have to search during the demo. (4) Record a 3-minute screen capture as a backup. (5) The advanced generator can take a couple of seconds, so trigger it early or use the bank.

7. Q&A bank: 30 likely questions

Big-picture

Q1. Where is the AI? There's no machine learning.

AI is more than ML. Constraint satisfaction, search, knowledge representation and rule-based inference are core AI (Russell & Norvig, Ch. 3, 6, 7 and 9). Our coach is an agent that reasons over a constraint model, chains inference rules, and explains its conclusions.

Q2. Why not just use ChatGPT as the tutor?

An LLM can invent a plausible-sounding but false deduction, and it has no guarantee of correctness. Every hint we give comes from a rule that provably fires on the current board, so it is correct by construction. It also costs nothing to run and works offline.

Q3. What is novel here?

Most Sudoku apps either reveal the answer or give generic tips. We (a) explain the actual deduction with the named technique and the cells that cause it, (b) grade and generate puzzles by the techniques they need, and (c) offer What-If lookahead so learners can explore hypotheses safely.

Q4. Who is the target user?

Learners from complete beginners (with the lessons) to advanced players (pairs, triples, X-Wing). AI students can also use it to see constraint propagation in action.

Q5. How is the coach adaptive?

The skill check or the user's choice sets a level. The level decides lessons versus puzzles, which puzzles are offered, and the highest technique tier the hint engine may use.

The AI techniques

Q6. How is Sudoku a CSP?

81 variables, domains {1..9}, and 27 AllDifferent constraints. Each cell must differ from its 20 peers.

Q7. What are candidates in AI terms?

The current **domain** of each variable after constraint propagation. Pencil marks are just the domains made visible.

Q8. Explain Norvig's two propagation rules.

(1) When a cell is fixed, remove its digit from all its peers. (2) If a unit has only one place left for a digit, put it there. They repeat until nothing changes.

Q9. Why try techniques from easiest to hardest?

It gives the explanation a human is most likely to find and understand. It is also cheaper, because simple rules are fast to check. This is the human version of preferring the most constrained choice.

Q10. What is forward chaining, and where do you use it?

Start from known facts (the board and its candidates), apply rules that match, add the new facts, and repeat. The hint ladder chains technique steps forward until one of them places a digit.

Q11. Why does the hint teach the hardest technique in the chain?

Because that is the step that made progress possible. The singles that follow are trivial, and naming the key step is what actually teaches something.

Q12. Explain a naked pair.

Two cells in a unit both have exactly the same 2 candidates. Those two digits must go in those two cells, so remove them from every other cell in the unit.

Q13. Explain a pointing pair.

Within a box, a digit can only appear in cells on one row (or column). It must be in that box's part of the line, so remove it from the rest of the line.

Q14. Explain X-Wing.
In two rows, a digit can only go in the same two columns. Whichever way it is placed, both columns are covered, so remove it from those columns in every other row.

Q15. What does What-If actually compute?
It assigns the value on a deep copy of the board, runs constraint propagation, and records the cascade of steps (marked *guessed*). If any cell runs out of candidates, that is a contradiction, and it reports which cell failed.

Q16. How does Check square know a move is a dead end (Case B)?
Our puzzles have a unique solution, which the CSP solver finds and the validator checks. Any digit that differs from it cannot be part of a valid completion, so it must lead to a contradiction eventually.

Q17. How does the solver work?
Norvig's agent: propagate with the two rules; if stuck, branch on the cell with the fewest candidates (MRV) and backtrack on contradiction. It takes about 2 ms on our puzzles.

Q18. What is MRV, and why is it good?
Minimum Remaining Values ("fail-first"): choose the variable with the smallest domain. It keeps branching small and exposes dead ends early.

Puzzles and implementation

Q19. How do you generate a puzzle with a unique solution?
Build a random complete grid, then blank cells in random order, keeping a removal only if our technique engine can still solve the puzzle by logic. Every technique step is a forced deduction, so if logic alone finishes the puzzle, it has exactly one solution.

Q20. How do you grade difficulty?
By the hardest technique the engine needs to finish the puzzle. An intermediate puzzle must *not* be solvable with singles alone. Clue count is only a rough indicator.

Q21. Why are advanced puzzles slower to generate?
Random digging rarely produces puzzles that require a naked triple or X-Wing. That is why a background pool keeps some pre-generated.

Q22. What if no technique applies?
The coach says that the standard techniques are exhausted and suggests What-If, which is the human form of a backtracking branch.

Q23. How fast is a hint?
About 2 ms median to compute a full Stage 3 hint, across all tiers of our bank.

Q24. How did you test it?
39 unit and integration tests: API endpoints, hint stages, technique detectors, generator uniqueness and grading, teaching quality, the validator and the solver.

Q25. Why FastAPI with a plain JS front end?
It is lightweight, the logic stays in Python next to the AI engine, and the UI is a thin client over a REST API.

Q26. Does What-If change the board?
No. It works on a deep copy, and the player's BoardState is never modified.

Critical-thinking

Q27. What are the limitations?
There is one shared board state per server (no multi-user sessions yet); the technique set stops at X-Wing (no Swordfish or chains); the skill check is a simple rule-based score; and we haven't yet run a user study to measure how much people learn.

Q28. How would you evaluate whether it teaches well?

Run a pre-test and post-test with two groups (coach versus answer-only hints), and compare solve time, hint usage and technique-recognition scores.

Q29. Where could ML fit in later?

Learning the user's skill from their move history (knowledge tracing), or predicting which hint stage a learner needs. The deductions themselves would stay symbolic so they remain correct.

Q30. Who built what?

Agree on this before the evaluation. Each member names their modules and can also explain one module that belongs to someone else.

8. Fix before the evaluation

#	Issue	Priority	Suggested fix
1	sudoku_ai_coach/README.md describes an older design: a 4-level "Socratic" escalation, Case B as forward checking, and an "AI Teacher Hub" with MRV quizzes. The UI now uses the 3-stage ladder (Nudge, Technique, Reveal), and Case B compares against the unique solution.	High	Update the README to match the app. An examiner reading it will spot the mismatch.
2	Old endpoints that the UI no longer calls: /api/hint, /api/explain, /api/analyze, /api/check-move (a duplicate of /api/teacher/check-move), plus /api/teacher/why-it-works.	Medium	Either remove them (along with coach_session/hint_engine/explanation_engine if they become unused) or call them "developer endpoints"
3	data/ is in .gitignore, so puzzle_bank.json and generated_pool.json aren't in git. Teammates' first launch will be slow while the bank rebuilds.	Medium	Remove data/ from .gitignore, or track just those 2 JSON files
4	The cleanup (deleted benchmark files) is not committed yet	Medium	Commit it so every teammate demos the same code
5	The generator test can fail when the machine is slow (it depends on a time budget)	Low	Don't run the test suite live; show it passing in a screenshot
6	There is one global board_state on the server, so two browser tabs share one game	Low	Demo in a single tab; list it as a known limitation

Rehearsal checklist

- Do 2 timed run-throughs. Finish at 14:00 to leave a buffer.
- Each member answers 8 random questions from Section 7 without notes.
- Pick **one demo puzzle** and note the exact cells for Case A, B and C and for What-If.
- Run the demo on the presenting laptop at projector resolution. Check the zoom so the 9×9 grid is readable from the back.
- Bring a pendrive with the slides as PDF, the demo recording, and the repository.

9. Presentation skills: easy marks

- **Open with the problem** ("hints that spoil vs hints that teach"), not with "Good morning, our project is..."
- **Keep slides light:** one idea per slide, at most 6 lines. The demo carries the weight.
- **Narrate the AI during the demo:** say the technical term each time ("this is forward chaining", "this is a domain wipe-out").
- **Hand over cleanly:** "...that's the AI inside. M3 will now show it live."
- **During Q&A:** pause, repeat the question, give the short answer first and the detail second. If you don't know, say what you would expect and why. Never bluff.
- **Let the person who owns a module answer questions about it**, and let one other member add a point.

The one-line pitch: "We built an adaptive Sudoku tutor whose every hint comes from a constraint-reasoning engine: nine human techniques as inference rules over CSP domains, chained forward to explain each move, with What-If lookahead and puzzles graded by the reasoning they require."